AGENTIC DARK POOL

Developer Work Assignment

Full-Stack Implementation Guide — Phase 0 through Phase 5

PROJECT	VERSION	DATE	CLASSIFICATION
Agentic Dark Pool	2.0	March 29, 2026	Internal Only

AGENTIC DARK POOL — INTERNAL DEVELOPMENT DOCUMENT — CONFIDENTIAL — VERSION 2.0 — MARCH 29, 2026

CONTENTS

Table of Contents

01 Work Split Overview

02 Shared Setup & Standards

03 Phase 0 — Sorrowz (Core Trading & Settlement)

04 Phase 0 — Powerz (Infrastructure & Services)

05 Phase 1 — Private Data Marketplace

06 Phase 2 — Private AI Inference Marketplace

07 Phase 3 — Private Perpetuals & Derivatives

08 Phase 4 — Privacy-as-a-Service SDK

09 Phase 5 — Agentic Economy Infrastructure

10 Shared Interfaces & Data Contracts

11 Master Timeline

12 Dependency Map

13 Token Utility Implementation

14 Base Ecosystem Integration Map

15 Revenue Projections

16 Appendix & Sign-off

SECTION 01

Work Split Overview

Complete responsibility matrix across all six phases of the Agentic Dark Pool expansion roadmap, split 50/50 between the two development leads.

Sorrowz

Core Trading & Settlement Lead

Phase 0 Core Dark Pool

- DarkPool.sol — commit-reveal order book
- Escrow.sol — USDC escrow & release
- SettlementVerifier.sol — ZK proof verification
- Orders Module — REST API for order lifecycle
- Matching Engine — batch auction clearing
- Settlement Service — on-chain settlement execution

Phase 1 Data Marketplace

- Private Bid Matching engine for data
- Federated Access Layer

Phase 2 AI Inference

- Encrypted Inference Jobs
- Private Inference Auctions
- SLA Enforcement contracts

Phase 3 Derivatives

- GPU-Hour Perpetuals contracts
- Compute Options

Powerz

Infrastructure & Services Lead

Phase 0 Core Dark Pool

- TokenRegistry.sol — GPU token registry
- ComputeCredit.sol — credit token
- FeeCollector.sol — protocol fee collection
- Auth Module — SIWE + API key system
- Provider Module — GPU provider management
- Market Module — price feeds & analytics
- WebSocket Server — real-time event stream
- Event Indexer — on-chain event tracking

Phase 1 Data Marketplace

- Encrypted Data Listings
- Data Provenance NFTs
- Differential Privacy Proofs

Phase 2 AI Inference

- Model Registry
- ZK Proof of Correct Execution verifier
- Validator Network

Phase 3 Derivatives

- Forward Contracts
- Cross-Margin engine

Phase 4 SDK

- ZK Order Commitment SDK Module
- Batch Auction as a Service (BaaS)

Phase 5 Agentic Economy

- Agent Dark Pool marketplace
- Multi-Agent Compute Coordination
- Proof of Agent Work

- ADP Compute Price Index (oracle)
- Insurance Fund Staking contract

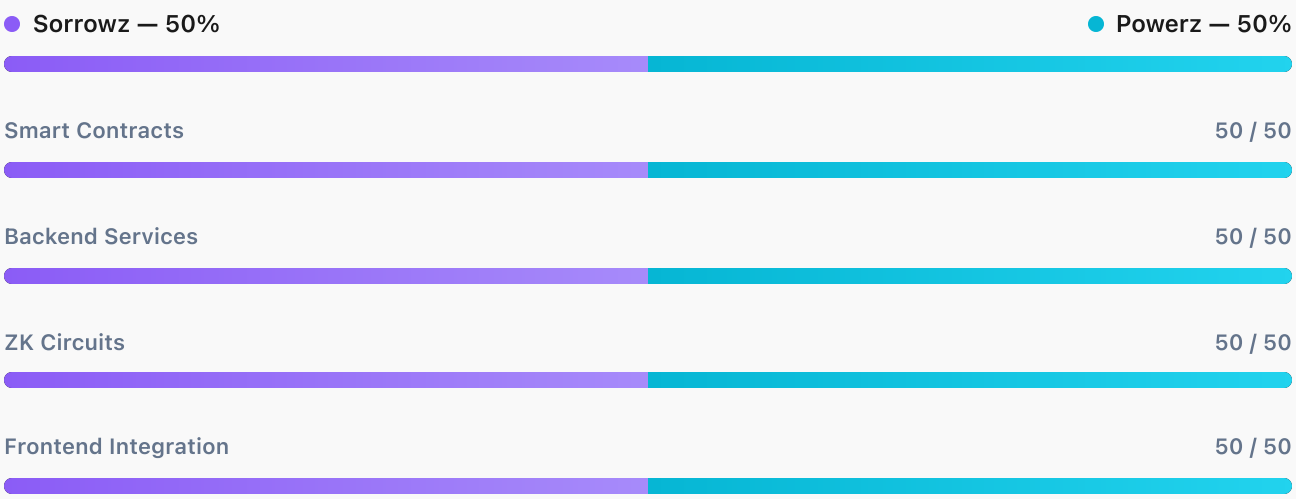
Phase 4 SDK

- TEE Compute Orchestration API
- ZK Proof Templates library
- Privacy Compliance Layer

Phase 5 Agentic Economy

- Private Agent Identity (ZK-based)
- Agent Credit System
- Agent Treasury Management

Workload Distribution



SECTION 02

Shared Setup & Standards

Common tooling, repository structure, and coding conventions that both developers must follow throughout all phases.

Monorepo Structure

```
// Root monorepo layout
agentic-dark-pool/
├── packages/
│   ├── contracts/          // Foundry project — Solidity ^0.8.24
│   │   ├── src/
│   │   ├── test/
│   │   ├── script/
│   │   └── foundry.toml
│   ├── backend/            // NestJS ^10.x application
│   │   ├── src/
│   │   │   ├── auth/
│   │   │   ├── orders/
│   │   │   ├── matching/
│   │   │   ├── settlement/
│   │   │   ├── providers/
│   │   │   ├── market/
│   │   │   ├── ws/
│   │   │   └── indexer/
│   │   └── prisma/schema.prisma
│   ├── frontend/           // Next.js / Vite + React
│   │   ├── src/
│   │   └── public/
│   └── zk-circuits/        // Noir circuits
│       ├── order_commitment/
│       ├── settlement_proof/
│       └── privacy_proofs/
├── .env.example
├── turbo.json
└── package.json
```

Tech Stack

LAYER	TECHNOLOGY	VERSION
Smart Contracts	Foundry / Solidity	^0.8.24
Backend Framework	NestJS	^10.x
Database	PostgreSQL	^16
Cache / Pub-Sub	Redis	^7
Blockchain Client	viem / wagmi	^2.x
Real-time	Socket.io	^4.x
ZK Proving	Noir	latest
Target Chain	Base (Sepolia / Mainnet)	—

Coding Standards

STANDARD	RULE
TypeScript	Strict mode enabled — `strict: true` in tsconfig
Formatting	Prettier (printWidth: 100, singleQuote: true, trailingComma: all)
Linting	ESLint with @typescript-eslint/recommended + plugin:prettier/recommended
Commits	Conventional Commits — `feat:` `fix:` `chore:` `docs:` `test:`
Branch Naming	`feat/<phase>-<module>` e.g. `feat/p0-matching-engine`
PR Reviews	Minimum 1 approval required; cross-review between Sorrowz and Powerz

STANDARD	RULE
Testing	Minimum 90% coverage on all contract functions; integration tests for all API endpoints

Environment Variables

```
# .env.example — shared across both developer environments
DATABASE_URL="postgresql://adp:adp@localhost:5432/darkpool"
REDIS_URL="redis://localhost:6379"
BASE_SEPOLIA_RPC="https://sepolia.base.org"
BASE_MAINNET_RPC="https://mainnet.base.org"
DEPLOYER_PRIVATE_KEY="0x..."
JWT_SECRET="generate-a-secure-random-string"
USDC_ADDRESS="0x036CbD53842c5426634e7929541eC2318f3dCF7e"
COINBASE_API_KEY="..."
COINBASE_API_SECRET="..."
```

SECTION 03

Phase 0 — Sorrowz Core Trading

Smart contracts for the encrypted order book and matching engine, plus the backend services that power order lifecycle and settlement.

DarkPool.sol Sorrowz

The core order book contract. Implements a commit-reveal scheme so order details remain hidden until the batch auction clears.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`commitOrder(bytes32 hash)`	external	Submit a hashed order commitment during the commit phase
`revealOrder(OrderParams calldata params, bytes32 salt)`	external	Reveal order details during the reveal phase; must match prior commitment
`cancelOrder(bytes32 orderId)`	external	Cancel an unmatched order; refunds escrowed USDC
`settleBatch(BatchResult calldata result, bytes calldata proof)`	external	Called by the settlement service; verifies ZK proof and executes settlement
`getCurrentBatch()`	view	Returns current batch ID, phase, and timestamps
`getOrder(bytes32 orderId)`	view	Returns order details (only available after reveal)

Commit-Reveal Scheme

```
// Solidity — commit-reveal pattern
struct OrderParams {
    address trader;
    OrderSide side;          // BUY or SELL
    GpuType gpuType;         // H100, A100, etc.
    uint256 quantity;        // GPU-hours
    uint256 pricePerUnit;    // in USDC (6 decimals)
    uint256 expiry;          // block.timestamp deadline
}

function commitOrder(bytes32 hash) external {
    require(currentPhase == Phase.COMMIT, "Not in commit phase");
    commitments[msg.sender][currentBatchId] = hash;
    emit OrderCommitted(msg.sender, currentBatchId, hash);
}

function revealOrder(
    OrderParams calldata params,
    bytes32 salt
) external {
    require(currentPhase == Phase.REVEAL, "Not in reveal phase");
    bytes32 hash = keccak256(abi.encode(params, salt));
    require(hash == commitments[msg.sender][currentBatchId], "Hash mismatch");
    orders[nextOrderId] = Order({ /* ... */ });
    emit OrderRevealed(nextOrderId, params.side, params.gpuType);
}
```

Events

EVENT	PARAMETERS
`OrderCommitted`	trader, batchId, commitHash
`OrderRevealed`	orderId, side, gpuType
`OrderCancelled`	orderId, trader
`BatchSettled`	batchId, clearingPrice, totalVolume

Testing Requirements

- Fuzz test commit-reveal with random salts and order parameters
- Invariant test: no order can be revealed without a prior commitment

- Test batch lifecycle: COMMIT → REVEAL → SETTLE → next batch
- Edge case: reveal with wrong salt must revert
- Gas benchmarks for settleBatch with 10, 50, 100 orders

Escrow.sol `Sorrowz`

Holds USDC during the order lifecycle. Funds are locked on commitment and released on settlement or cancellation.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`deposit(uint256 amount)`	external	Lock USDC into escrow for an order
`release(address to, uint256 amount)`	onlyDarkPool	Release USDC to seller after successful settlement
`refund(address to, uint256 amount)`	onlyDarkPool	Refund USDC on cancellation or failed settlement
`balanceOf(address user)`	view	Returns escrowed balance for a user

Testing Requirements

- Only DarkPool.sol can call release and refund (access control)
- Deposit/release accounting must always balance
- Invariant: total escrowed equals sum of all individual balances
- Reentrancy protection test

SettlementVerifier.sol `Sorrowz`

Verifies ZK proofs that the batch auction was settled correctly. Ensures the clearing price is valid and all matched orders are legitimate.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`verify(bytes calldata proof, bytes32[] calldata publicInputs)`	view	Verify a Noir-generated settlement proof on-chain
`setVerifierAddress(address newVerifier)`	onlyOwner	Update the UltraPlonk verifier contract address
`verifierAddress()`	view	Return the current verifier contract

Testing Requirements

- Valid proof must return true
- Invalid/tampered proof must return false
- Gas benchmark for proof verification

Orders Module (Backend) `Sorrowz`

NestJS module providing the REST API for the full order lifecycle: submission, status, cancellation, and history retrieval.

API Endpoints

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/orders`	Submit a new order (generates commitment, stores locally)
GET	`/api/orders/:id`	Retrieve order status and details
GET	`/api/orders`	List orders with filters (status, side, gpuType)
DELETE	`/api/orders/:id`	Cancel a pending order
GET	`/api/orders/:id/settlement`	Get settlement details for a filled order

Database Schema — orders

```sql
-- PostgreSQL schema for orders table
CREATE TABLE orders (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID           NOT NULL REFERENCES users(id),
  side            VARCHAR(4)     NOT NULL,  -- BUY | SELL
  gpu_type        VARCHAR(10)    NOT NULL,  -- H100 | A100 | L40S
  quantity        DECIMAL(18,6)  NOT NULL,
  price_per_unit  DECIMAL(18,6)  NOT NULL,
  status          VARCHAR(12)    NOT NULL DEFAULT 'PENDING',
  commit_hash     BYTEA,
  batch_id        INTEGER,
  salt            BYTEA,
  created_at      TIMESTAMPTZ    DEFAULT now(),
  updated_at      TIMESTAMPTZ    DEFAULT now()
);
```

Matching Engine (Backend) Sorrowz

Implements the batch auction clearing algorithm. Runs on a timer (every 30-60 seconds) to match buy and sell orders at a uniform clearing price.

Batch Auction Algorithm

```typescript
// Pseudo-code: Uniform Price Batch Auction
function clearBatch(bids: Order[], asks: Order[]): BatchResult {
  // 1. Sort bids descending by price, asks ascending
  bids.sort((a, b) => b.price - a.price);
  asks.sort((a, b) => a.price - b.price);

  // 2. Build aggregate supply/demand curves
  let demandCurve = buildCumulativeCurve(bids);
  let supplyCurve = buildCumulativeCurve(asks);

  // 3. Find intersection — the clearing price
  let clearingPrice = findIntersection(demandCurve, supplyCurve);

  // 4. Match all bids >= clearingPrice with asks <= clearingPrice
  let matchedBids = bids.filter(b => b.price >= clearingPrice);
  let matchedAsks = asks.filter(a => a.price <= clearingPrice);
```

```
  // 5. Pro-rata fill if supply != demand at clearing price
  let fills = proRataFill(matchedBids, matchedAsks, clearingPrice);

  // 6. Generate ZK proof of correct settlement
  let proof = await generateSettlementProof(fills, clearingPrice);

  return { clearingPrice, fills, proof, batchId };
}
```

Database Schema — settlements

```
CREATE TABLE settlements (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id        INTEGER        NOT NULL UNIQUE,
  clearing_price  DECIMAL(18,6)  NOT NULL,
  total_volume    DECIMAL(18,6)  NOT NULL,
  num_fills       INTEGER        NOT NULL,
  proof           BYTEA          NOT NULL,
  tx_hash         BYTEA,
  status          VARCHAR(12)    NOT NULL DEFAULT 'PENDING',
  settled_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ    DEFAULT now()
);
```

Settlement Service (Backend) `Sorrowz`

Orchestrates the on-chain settlement: submits ZK proof, triggers escrow release, and tracks transaction confirmation.

Responsibilities

- Listen for completed batch auction results from the matching engine
- Call `DarkPool.settleBatch()` with the ZK proof and batch result
- Monitor transaction status and handle retries on failure
- Update order statuses to FILLED or PARTIALLY_FILLED
- Publish settlement events to Redis for the WebSocket server to relay

```sql
CREATE TABLE compute_jobs (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  settlement_id   UUID           REFERENCES settlements(id),
  buyer_id        UUID           NOT NULL REFERENCES users(id),
  seller_id       UUID           NOT NULL REFERENCES users(id),
  gpu_type        VARCHAR(10)    NOT NULL,
  quantity        DECIMAL(18,6)  NOT NULL,
  price_per_unit  DECIMAL(18,6)  NOT NULL,
  status          VARCHAR(16)    NOT NULL DEFAULT 'ALLOCATED',
  started_at      TIMESTAMPTZ,
  completed_at    TIMESTAMPTZ,
  created_at      TIMESTAMPTZ    DEFAULT now()
);
```

Frontend Integration `Sorrowz`

Wire order flow pages to real API and contracts. Sorrowz owns the frontend for trading-specific pages.

- **Order Submission Page** — Connect form to `POST /api/orders`, call `DarkPool.commitOrder()` via wagmi
- **Order Book View** — Real-time display of current batch orders (anonymized) via WebSocket
- **My Orders Page** — List user orders with status badges, cancel action, settlement details
- **Batch Auction Timer** — Visual countdown showing COMMIT → REVEAL → SETTLE phases
- **Settlement History** — Table of past settlements with clearing prices, volumes, and on-chain tx links

SECTION 04

Phase 0 — Powerz Infrastructure

Supporting smart contracts, authentication, provider management, market data, real-time events, and on-chain indexing.

TokenRegistry.sol Powerz

On-chain registry mapping GPU types to standardized token IDs. Ensures consistent naming and metadata across the protocol.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`registerGpuType(string name, uint256 tokenId, GpuMeta meta)`	onlyOwner	Register a new GPU type with metadata
`deactivateGpuType(uint256 tokenId)`	onlyOwner	Mark a GPU type as inactive (no new orders)
`getGpuType(uint256 tokenId)`	view	Return GPU type metadata
`getAllActiveGpuTypes()`	view	Return all active GPU type IDs and names
`isValidGpuType(uint256 tokenId)`	view	Check if a GPU type is registered and active

GPU Types to Register

TOKEN ID	NAME	VRAM	TIER
1	NVIDIA H100 SXM	80 GB	Premium
2	NVIDIA A100 SXM	80 GB	Standard
3	NVIDIA L40S	48 GB	Standard

TOKEN ID	NAME	VRAM	TIER
4	NVIDIA H200	141 GB	Premium
5	NVIDIA A10G	24 GB	Economy

ComputeCredit.sol Powerz

ERC-20 token representing compute credits. Users can purchase credits with USDC to streamline repeat transactions.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`mint(address to, uint256 amount)`	onlyMinter	Mint credits in exchange for USDC deposit
`burn(address from, uint256 amount)`	onlyDarkPool	Burn credits when used for order payment
`exchangeRate()`	view	Returns current USDC:credit exchange rate

FeeCollector.sol Powerz

Collects protocol fees from every settled batch. Distributes fees between treasury, stakers, and burn.

Function Table

FUNCTION	VISIBILITY	DESCRIPTION
`collectFee(uint256 amount, bytes32 batchId)`	onlyDarkPool	Collect fee from a settled batch

FUNCTION	VISIBILITY	DESCRIPTION
`distributeFees()`	external	Distribute accumulated fees to treasury, stakers, and burn address
`setFeeRate(uint256 bps)`	onlyOwner	Set fee rate in basis points (default: 20 bps = 0.2%)
`setDistribution(uint256 treasuryBps, uint256 stakerBps, uint256 burnBps)`	onlyOwner	Set fee distribution split
`totalCollected()`	view	Total fees collected since deployment

Deployment Script Order

```
// script/Deploy.s.sol — deployment order matters
1. Deploy TokenRegistry
2. Deploy ComputeCredit
3. Deploy FeeCollector
4. Deploy Escrow                  // needs USDC address
5. Deploy SettlementVerifier      // needs Noir verifier
6. Deploy DarkPool               // needs all above addresses
7. Configure cross-references   // set DarkPool addr on Escrow, FeeCollector
```

Auth Module (Backend) Powerz

Sign-In with Ethereum (SIWE) authentication flow plus API key generation for programmatic access.

SIWE Authentication Flow

STEP	ENDPOINT	DESCRIPTION
1	`GET /api/auth/nonce`	Generate a random nonce for the wallet to sign
2	`POST /api/auth/verify`	Verify the signed SIWE message, issue JWT

STEP	ENDPOINT	DESCRIPTION
3	`POST /api/auth/refresh`	Refresh an expiring JWT
4	`POST /api/auth/api-keys`	Generate a new API key for the authenticated user
5	`DELETE /api/auth/api-keys/:id`	Revoke an API key

Database Schemas

```
CREATE TABLE users (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  wallet_address  VARCHAR(42)    NOT NULL UNIQUE,
  nonce           VARCHAR(64)    NOT NULL,
  role            VARCHAR(10)    NOT NULL DEFAULT 'TRADER',
  created_at      TIMESTAMPTZ    DEFAULT now(),
  updated_at      TIMESTAMPTZ    DEFAULT now()
);

CREATE TABLE api_keys (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID           NOT NULL REFERENCES users(id),
  key_hash        BYTEA          NOT NULL UNIQUE,
  label           VARCHAR(64),
  permissions     JSONB          NOT NULL DEFAULT '["read","trade"]',
  expires_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ    DEFAULT now()
);
```

NestJS Guards to Export

- `JwtAuthGuard` — validates JWT from Authorization header
- `ApiKeyGuard` — validates X-API-Key header
- `RolesGuard` — checks user role against required roles decorator
- `WalletOwnerGuard` — ensures the request wallet matches the resource owner

Provider Module (Backend) `Powerz`

Manages GPU compute providers: registration, capacity reporting, uptime tracking, and reputation scoring.

API Endpoints

METHOD	ENDPOINT	DESCRIPTION
POST	`/api/providers`	Register as a compute provider
GET	`/api/providers`	List all active providers (public)
GET	`/api/providers/:id`	Get provider details and stats
PATCH	`/api/providers/:id/capacity`	Update available capacity
GET	`/api/providers/:id/reputation`	Get reputation score and history

Database Schema — providers

```
CREATE TABLE providers (
  id              UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID           NOT NULL REFERENCES users(id),
  name            VARCHAR(100)   NOT NULL,
  gpu_types       JSONB          NOT NULL,  -- [{type, count, available}]
  region          VARCHAR(32),
  uptime_pct      DECIMAL(5,2)   DEFAULT 100.00,
  reputation      DECIMAL(5,2)   DEFAULT 100.00,
  total_jobs      INTEGER        DEFAULT 0,
  status          VARCHAR(10)    NOT NULL DEFAULT 'ACTIVE',
  created_at      TIMESTAMPTZ    DEFAULT now(),
  updated_at      TIMESTAMPTZ    DEFAULT now()
);
```

Market Module (Backend) Powerz

Aggregates price data and analytics. Serves as the frontend data source for market overview and price charts.

API Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/market/prices`	Current spot prices per GPU type
GET	`/api/market/prices/history`	Historical clearing prices (OHLCV)
GET	`/api/market/volume`	24h/7d/30d volume by GPU type
GET	`/api/market/stats`	Global stats: TVL, total trades, avg clearing price

Data Sources

- Settlement records from the settlements table (clearing prices)
- Order records for demand/supply metrics
- Provider capacity data for utilization metrics
- Cached in Redis with 10-second TTL for high-frequency reads

WebSocket Server (Backend) Powerz

Real-time event streaming via Socket.io. Clients subscribe to channels and receive push updates for all protocol activity.

WebSocket Events

EVENT	CHANNEL	PAYLOAD
`batch:phase`	public	{ batchId, phase, endsAt }
`batch:settled`	public	{ batchId, clearingPrice, volume, txHash }
`order:status`	user-scoped	{ orderId, status, fillPrice, fillQty }
`market:price`	public	{ gpuType, price, change24h }
`provider:status`	admin	{ providerId, status, capacity }

Implementation

- Redis subscriber listens to `adp:events:*` channels
- NestJS WebSocket gateway forwards events to connected clients
- JWT authentication on WebSocket handshake
- Heartbeat ping every 30 seconds to maintain connections

Event Indexer (Backend) Powerz

Listens to on-chain events from all deployed contracts and indexes them into PostgreSQL for fast querying.

Events to Index

CONTRACT	EVENT	INDEXED FIELDS
DarkPool	OrderCommitted	trader, batchId
DarkPool	OrderRevealed	orderId, side, gpuType
DarkPool	BatchSettled	batchId, clearingPrice
Escrow	Deposited	user, amount
Escrow	Released	user, amount
FeeCollector	FeeCollected	batchId, amount
FeeCollector	FeesDistributed	treasury, stakers, burned

Implementation

- Use `viem.watchContractEvent()` for real-time event subscription
- Persist block cursor in Redis for crash recovery
- Batch insert events into PostgreSQL every 5 seconds
- Publish to Redis pub/sub for the WebSocket server to relay

Frontend Integration Powerz

Wire infrastructure and service pages to real APIs and WebSocket feeds.

- **Market Overview Page** — Connect to `GET /api/market/*` endpoints, live price tickers via WebSocket
- **Provider Dashboard** — Connect to `/api/providers`, capacity cards, reputation display
- **Settings Page** — API key management UI, wallet connection, notification preferences
- **Live Feed** — Real-time activity stream from WebSocket events (settlements, new providers, price changes)
- **Auction Timer Component** — Shared component displaying current batch phase with countdown, subscribing to `batch:phase` WebSocket event

SECTION 05

Phase 1 — Private Data Marketplace

Q1–Q2 2027

Extending the dark pool primitives to support private trading of datasets and data access rights.

Data Bid Matching Engine

Sorrowz

Extend the batch auction engine to support data marketplace orders. Data auctions have longer batch intervals to account for the different nature of dataset trading.

Key Differences from Compute Auctions

- Batch auctions clear every **10 minutes** (vs 30-60 seconds for compute)
- New order type: `DATA_BID` with fields for dataset category, format requirements, size range, and max price
- Matching considers data quality scores and provider reputation in addition to price
- Support for partial dataset access (subsets of a larger dataset)

New Order Fields

```
interface DataBidOrder {
  type: 'DATA_BID';
  category: DataCategory;       // HEALTHCARE | FINANCE | NLP | VISION | etc.
  formatRequirements: string[]; // ['parquet', 'csv']
  sizeRange: {
    min: number;               // minimum GB
    max: number;               // maximum GB
  };
  maxPrice: bigint;             // USDC (6 decimals)
  qualityThreshold: number;     // minimum quality score 0-100
}
```

Federated Access Layer

Sorrowz

Build the access token system that governs data access after bid matching. Computation runs at the data source inside TEE, so the data never leaves the provider.

DataAccessToken.sol

ERC-721 token representing time-limited access rights to run computation at the data source inside a TEE.

FUNCTION	DESCRIPTION
`mint(address buyer, uint256 datasetId, uint256 expiry)`	Mint an access token after successful bid match
`isValid(uint256 tokenId)`	Check if token is still within its access window
`revoke(uint256 tokenId)`	Provider can revoke on SLA violation
`extend(uint256 tokenId, uint256 newExpiry)`	Extend access period with additional payment

Settlement Extension

Extend Escrow.sol to handle data marketplace settlements with longer escrow periods. Data delivery verification takes longer than compute verification, so escrow windows are 24-72 hours rather than minutes.

Encrypted Data Listings Powerz

Backend module for data providers to register datasets. ZK proofs verify dataset properties without revealing the data itself.

Responsibilities

- REST API for data providers to register datasets with encrypted metadata
- ZK proofs verify dataset properties: size, format, domain category, and quality score
- Store dataset metadata hashes on-chain via Base transaction
- Search and discovery API for data buyers to find relevant datasets by category and properties
- Integration with TEE for secure data preview (statistical summaries only)

Data Provenance NFTs Powerz

Smart contract for tokenized dataset ownership with automatic royalty distribution.

DataProvenance.sol

ERC-721 with ERC-2981 royalty support. Each dataset is tokenized as a verifiable on-chain asset.

FUNCTION	DESCRIPTION
`mintDataset(address provider, bytes32 metadataHash, uint96 royaltyBps)`	Tokenize a verified dataset
`royaltyInfo(uint256 tokenId, uint256 salePrice)`	ERC-2981 royalty calculation
`updateMetadataHash(uint256 tokenId, bytes32 newHash)`	Update dataset version
`accessCount(uint256 tokenId)`	Total access events for a dataset

Automatic royalty flow to data providers on every access event. Minted via OnchainKit on Base for streamlined user experience.

Differential Privacy Proofs Powerz

Noir ZK circuits that prove a dataset meets differential privacy standards. Critical for healthcare and finance datasets where regulatory compliance is mandatory.

- Noir circuit proves epsilon-delta differential privacy guarantees
- Verifier contract deployed on Base for on-chain verification
- Privacy budget tracking per dataset to prevent over-querying
- Integration with Data Provenance NFT metadata

Token Utility Integration

- Staking tiers for data access: higher stake unlocks premium datasets
- Provider reward distribution proportional to dataset usage and quality scores

SECTION 06

Phase 2 — Private AI Inference Marketplace

Q3–Q4 2027

A private marketplace for AI inference capacity. Buyers request inference, providers offer capacity, and ZK proofs guarantee correct execution.

Encrypted Inference Jobs `Sorrowz`

Build the inference job submission system. Buyers submit inference requests encrypted to a TEE public key, ensuring input data confidentiality.

InferencePool.sol

Smart contract managing the encrypted job queue, escrow for inference payments, and SLA enforcement.

FUNCTION	DESCRIPTION
`submitJob(bytes encryptedInput, uint256 modelId, uint256 maxPrice)`	Submit an encrypted inference request
`claimJob(uint256 jobId)`	Provider claims a job from the queue
`completeJob(uint256 jobId, bytes encryptedOutput, bytes proof)`	Submit result with ZK proof of correct execution
`disputeJob(uint256 jobId)`	Buyer disputes a completed job within the dispute window
`refundJob(uint256 jobId)`	Auto-refund if SLA is violated

Private Inference Auctions `Sorrowz`

Extend the batch auction engine for inference capacity matching. Buyers bid on inference capacity without revealing their workload details.

- Bid parameters: tokens/second throughput, model type, max latency requirement, batch size
- Providers offer capacity without revealing current utilization levels
- Matching considers model availability, provider latency history, and price
- Batch intervals: 2-5 minutes for inference (lower latency requirements than data marketplace)

SLA Enforcement Contracts `Sorrowz`

Smart contracts that hold USDC in escrow and release payment only when ZK proof of correct execution is verified on Base.

- Escrow locks USDC on job submission
- Payment released only after ZK proof verification passes on-chain
- Automatic refunds if SLA is violated: latency exceeded, wrong model used, output malformed
- Slashing mechanism for providers who repeatedly violate SLA
- Dispute resolution with configurable arbitration window (default: 4 hours)

Model Registry `Powerz`

On-chain registry where AI model developers list both open-source and proprietary models. Models are encrypted and only executable inside verified TEEs.

ModelRegistry.sol

FUNCTION	DESCRIPTION
`registerModel(ModelMeta calldata meta)`	Register a new model with metadata
`updateModel(uint256 modelId, ModelMeta calldata meta)`	Update model metadata (new version, benchmarks)
`getModel(uint256 modelId)`	Retrieve model metadata

FUNCTION	DESCRIPTION
`listModels(ModelFilter calldata filter)`	Search models by type, GPU requirements, benchmark scores

Model Metadata

```
struct ModelMeta {
    string name;              // "Llama-3-70B"
    string version;           // "1.0.0"
    bytes32 inputSchemaHash;  // hash of expected input format
    bytes32 outputSchemaHash; // hash of output format
    GpuType requiredGpu;      // minimum GPU tier required
    uint256 benchmarkScore;   // tokens/second on reference hardware
    bool isOpenSource;        // licensing flag
    address developer;        // royalty recipient
}
```

ZK Proof of Correct Execution Powerz

Noir circuit plus Solidity verifier that proves the correct model was run on the submitted input without revealing either the model weights or the input data.

- TEE generates the ZK proof after completing inference
- Proof attests: correct model ID was loaded, input hash matches submission, output is deterministic for given input
- Verifier contract deployed on Base, callable by SLA Enforcement contracts
- Proof generation target: under 10 seconds for standard inference jobs

Validator Network Powerz

Backend service for ZK proof verification nodes. Token stakers run verification infrastructure and earn fees per verified inference job.

- Stakers register as verification nodes by staking minimum 10,000 tokens

- Earn proportional share of verification fees per validated proof
- Slashing for incorrect verification: 10% of stake per false validation
- Round-robin job assignment with performance-based weighting
- Minimum 3-of-5 validator consensus for proof acceptance

Coinbase AgentKit Integration

Enable AI agents to autonomously purchase inference capacity via AgentKit and x402 payment protocol. Agents can submit inference jobs, manage USDC balances, and handle settlement without human intervention.

SECTION 07

Phase 3 — Private Perpetuals & Derivatives

Q1–Q2 2028

Financial derivatives on GPU compute prices. Perpetuals, options, and forward contracts settled on Base.

GPU-Hour Perpetuals Sorrowz

Perpetual futures contracts on H100 GPU compute prices. Encrypted order book using ZK commitments with a funding rate mechanism.

- Encrypted order book using the same commit-reveal ZK scheme from Phase 0
- Funding rate calculated every 8 hours based on deviation from ADP spot price index
- Maximum leverage: 10x with auto-deleveraging at 5% maintenance margin
- Liquidation engine with encrypted position management — liquidators cannot see position details until liquidation triggers
- Settlement in USDC on Base, with immediate margin updates

Compute Options Sorrowz

Call and put options on GPU compute prices. European-style with batch settlement.

- Strike prices set at standard intervals (e.g., $1.50, $2.00, $2.50 per GPU-hour)
- Weekly and monthly expiry cycles
- Premiums settled in USDC on Base at time of purchase
- European-style: exercise only at expiry, with batch settlement to reduce gas costs
- Black-Scholes-inspired pricing model adapted for compute volatility

Forward Compute Contracts Sorrowz

Tokenized forward contracts for future GPU capacity. AI companies lock in compute at today's prices.

- ERC-20 tokens representing forward positions — fully transferable on Base
- Standard maturities: 1 month, 3 months, 6 months, 12 months
- Physical delivery option: forward holder can claim actual GPU time at expiry
- Cash settlement option: settle the difference between forward price and spot at expiry
- Secondary market for trading forward positions before expiry

Cross-Margin Engine `Sorrowz`

Allow traders to use spot compute positions as collateral for derivatives. Full composability with the core dark pool.

- Spot compute positions valued at mark-to-market for collateral purposes
- Cross-margin calculation engine considers all positions: spot, perps, options, forwards
- Portfolio margining reduces total collateral requirements by up to 40%
- Real-time margin monitoring with WebSocket updates to traders
- Graceful liquidation: partial position closing before full liquidation

ADP Compute Price Index `Powerz`

Decentralized, manipulation-resistant GPU compute price oracle. The "Bloomberg terminal" for AI compute.

- Aggregate clearing prices from all batch auctions across the protocol
- Time-weighted average price (TWAP) calculation with outlier rejection
- Published on-chain via Base data feeds, readable by any smart contract
- Update frequency: every batch settlement (30-60 seconds for compute, 10 minutes for data)
- Historical data API for analytics, charting, and derivatives pricing models
- Feeds into all derivatives pricing: perpetual funding rates, option premiums, forward curves

Insurance Fund `Powerz`

Backstop for derivatives positions. Absorbs losses when liquidated positions go below zero.

InsuranceFund.sol

FUNCTION	DESCRIPTION
`stake(uint256 amount)`	Stake tokens into the insurance fund
`unstake(uint256 amount)`	Withdraw staked tokens (7-day cooldown)
`coverLoss(uint256 amount)`	Called by liquidation engine to cover shortfalls
`distributeFees()`	Distribute accumulated liquidation fees to stakers
`fundHealth()`	Return current fund size vs outstanding risk

- Token stakers provide backstop capital and earn share of liquidation fees
- Automatic recapitalization from protocol fees when fund drops below threshold

Institutional Integration

- Coinbase Prime integration for institutional derivatives access with custody solutions
- Base lending protocol integration for collateral composability (use LP tokens as margin)

Liquidity Mining Program

Token reward distribution for market makers providing liquidity to compute perpetuals. Rewards weighted by volume contribution and time in market.

SECTION 08

Phase 4 — Privacy-as-a-Service SDK

Q3 2028 – Q2 2029

Package core dark pool primitives as reusable SDK modules. Any protocol can integrate encrypted order books, batch auctions, and TEE computation.

ZK Order Commitment SDK Module

Sorrowz

Package the encrypted order book as a drop-in module for any DEX or marketplace.

- Published as npm package: `@adp/zk-orders`
- One SDK call encrypts an order, commits on-chain, and submits to batch auction
- Framework-agnostic: works with any EVM-compatible chain
- TypeScript-first with full type safety

```
// Example SDK usage
import { ZkOrderClient } from '@adp/zk-orders';

const client = new ZkOrderClient({
  rpcUrl: 'https://mainnet.base.org',
  contractAddress: '0x...',
});

// Single call: encrypt, commit, and submit
const receipt = await client.submitOrder({
  side: 'BUY',
  asset: 'H100',
  quantity: 10,
  maxPrice: parseUnits('2.50', 6),
});
```

Batch Auction as a Service (BaaS)

Sorrowz

Package the matching engine as a hosted API service. Any protocol can use it for their own asset class.

- REST API + WebSocket for full batch lifecycle management
- Configurable batch intervals (seconds to hours)
- Pluggable clearing algorithms: uniform price, discriminatory, sealed-bid
- Configurable fee structures with revenue share to ADP token holders
- Use cases: NFT auctions, prediction markets, tokenized RWAs, carbon credits

```
// BaaS API — create a new auction pool
POST /api/baas/pools
{
  "name": "Carbon Credit Auction",
  "asset": "VER-2027",
  "batchInterval": 300,         // 5 minutes
  "clearingAlgorithm": "uniform",
  "feeRate": 30,               // 0.3% in bps
  "settlementChain": "base"
}
```

TEE Compute Orchestration API Powerz

REST API for any dApp to route sensitive computation through ADP's TEE network.

- Published as SDK: `@adp/tee-compute`
- Endpoints for job submission, status polling, and encrypted result retrieval
- Auto-scaling TEE node management with load balancing
- Support for custom computation containers (Docker-in-TEE)

```
// TEE Compute SDK usage
import { TeeClient } from '@adp/tee-compute';

const tee = new TeeClient({ apiKey: 'adp_...' });

const job = await tee.submit({
  container: 'my-private-ml-model:latest',
  encryptedInput: inputBuffer,
  gpuType: 'H100',
  maxDuration: 3600,
});
```

```
const result = await tee.getResult(job.id);
```

ZK Proof Templates Powerz

Pre-built Noir ZK circuits for common privacy patterns. Published as a circuit library.

- Published as: `@adp/zk-circuits`
- **Private Balance Proofs** — prove you hold at least X tokens without revealing exact balance
- **Private Order Commitments** — commit to order parameters without revealing them
- **Private Identity Verification** — prove KYC status without revealing personal data
- **Private Dataset Proofs** — prove dataset properties without revealing contents
- Each circuit includes: Noir source, compiled ACIR, Solidity verifier, TypeScript bindings

Privacy Compliance Layer Powerz

ZK proofs that satisfy regulatory requirements without revealing underlying data. Enterprises prove compliance to regulators without exposing transaction details.

- AML compliance proofs: prove transaction source is not from sanctioned addresses
- KYC verification: prove identity verification status without revealing identity documents
- Tax reporting: prove tax obligations are met without revealing full transaction history
- Integration with Coinbase Verifications for attestation-based identity
- Configurable per jurisdiction: EU (MiCA), US (SEC/CFTC), Singapore (MAS)

SDK Documentation Portal

Developer onboarding experience including interactive tutorials, API reference, code examples, and integration guides. Hosted on Base Developer Docs infrastructure.

SECTION 09

Phase 5 — Agentic Economy Infrastructure

Q3 2029+

Infrastructure for autonomous AI agents to transact, coordinate, and build reputation privately on Base.

Agent Dark Pool

Sorrowz

Private marketplace where AI agents negotiate and transact with each other. All routed through encrypted batch auctions.

- Agent-to-agent transactions for services, data, compute, and capabilities
- Extension of the core matching engine with agent-specific order types
- Agent intent matching: agents express high-level goals, the matching engine finds optimal counterparties
- Multi-hop transaction routing for complex agent workflows
- Privacy-preserving reputation signals included in matching (via ZK proofs)

Multi-Agent Compute Coordination

Sorrowz

Orchestration layer for agent swarms to privately coordinate GPU cluster allocation.

AgentCoordinator.sol

FUNCTION	DESCRIPTION
`createSwarm(address[] agents, bytes32 goalHash)`	Register a coordinating agent swarm
`submitCollectiveBid(uint256 swarmId, bytes encryptedBid)`	Submit a collective bid on behalf of the swarm
`allocateResources(uint256 swarmId, AllocationPlan plan)`	Distribute acquired compute across swarm members

FUNCTION	DESCRIPTION
`settleSwarm(uint256 swarmId)`	Final settlement and cost splitting

Collective bidding without revealing individual agent strategies. Agents cooperate on procurement while maintaining competitive privacy.

Proof of Agent Work Sorrowz

ZK proofs that an agent completed a task correctly without revealing task details or methodology.

- Noir circuit proves: task was completed, output matches specification, agent had sufficient capability
- Solidity verifier on Base for on-chain verification
- Enables trustless agent-to-agent service markets
- Proof includes: task commitment hash, capability attestation, output commitment, execution timestamp
- Composable with Agent Credit System for reputation building

Private Agent Identity Powerz

ZK-based agent identity system on Base. Agents prove capabilities and reputation without revealing underlying model, owner, or transaction history.

AgentIdentity.sol

FUNCTION	DESCRIPTION
`registerAgent(bytes32 capabilityHash, bytes proof)`	Register an agent with ZK-proven capabilities
`verifyCapability(uint256 agentId, bytes32 capability)`	Verify an agent has a specific capability without revealing others
`updateReputation(uint256 agentId, bytes proof)`	Update reputation with ZK proof of completed work

FUNCTION	DESCRIPTION
`revokeAgent(uint256 agentId)`	Revoke agent identity on proven malicious behavior

Built via Coinbase Verifications and ERC-8004 agent identity standard. Interoperable with the broader Base agentic ecosystem.

Agent Credit System Powerz

On-chain credit scores based on verified transaction history. Credit scores are ZK-proven and not publicly visible.

AgentCredit.sol

FUNCTION	DESCRIPTION
`getScore(uint256 agentId)`	Returns encrypted credit score (only readable by authorized parties)
`proveMinScore(uint256 agentId, uint256 threshold, bytes proof)`	ZK prove credit score exceeds threshold
`recordTransaction(uint256 agentId, bytes32 txHash, bool success)`	Record a transaction outcome for credit scoring
`getCreditLimit(uint256 agentId)`	Maximum transaction value based on credit score

Higher credit scores unlock higher-value transactions and lower collateral requirements. Incentivizes good agent behavior over time.

Agent Treasury Management Powerz

AI agents autonomously manage USDC treasuries on Base. Earning yield, paying for compute, and settling contracts.

- Integration with x402 payment protocol for automated micropayments

- Integration with ADP privacy layer for confidential treasury operations
- Yield optimization: agents automatically allocate idle USDC to Base DeFi protocols
- Budget management: configurable spending limits and approval thresholds
- Multi-sig support for high-value transactions requiring human approval

x402 + AgentKit Deep Integration

ADP positioned as the default privacy middleware for the Base agentic economy. Every agent-to-agent transaction can optionally route through the dark pool for price privacy and MEV protection.

SECTION 10

Shared Interfaces & Data Contracts

TypeScript type definitions shared across all backend and frontend packages. Both developers import from `@adp/types`.

Order Types Shared

```
// packages/types/src/orders.ts

export enum OrderSide {
  BUY = 'BUY',
  SELL = 'SELL',
}

export enum OrderStatus {
  PENDING = 'PENDING',
  COMMITTED = 'COMMITTED',
  REVEALED = 'REVEALED',
  MATCHED = 'MATCHED',
  FILLED = 'FILLED',
  PARTIALLY_FILLED = 'PARTIALLY_FILLED',
  CANCELLED = 'CANCELLED',
  EXPIRED = 'EXPIRED',
}

export enum GpuType {
  H100 = 'H100',
  A100 = 'A100',
  L40S = 'L40S',
  H200 = 'H200',
  A10G = 'A10G',
}

export interface CreateOrderDto {
  side: OrderSide;
  gpuType: GpuType;
  quantity: number;
  pricePerUnit: string;  // USDC amount as string for precision
}
```

```
export interface OrderResponse {
  id: string;
  side: OrderSide;
  gpuType: GpuType;
  quantity: number;
  pricePerUnit: string;
  status: OrderStatus;
  commitHash: string | null;
  batchId: number | null;
  fillPrice: string | null;
  fillQuantity: number | null;
  createdAt: string;
  updatedAt: string;
}
```

Settlement & Market Types Shared

```
// packages/types/src/settlement.ts

export interface BatchResult {
  batchId: number;
  clearingPrice: string;
  totalVolume: number;
  numFills: number;
  proof: string;  // hex-encoded ZK proof
  txHash: string | null;
  settledAt: string | null;
}

export interface SettlementEvent {
  batchId: number;
  clearingPrice: string;
  totalVolume: number;
  txHash: string;
  timestamp: number;
}

// packages/types/src/market.ts

export interface MarketPrice {
  gpuType: GpuType;
```

```
  price: string;
  change24h: number;
  volume24h: string;
}

export interface ProviderInfo {
  id: string;
  name: string;
  gpuTypes: { type: GpuType; count: number; available: number }[];
  region: string;
  uptimePct: number;
  reputation: number;
}
```

Data Marketplace Types Shared

```
// packages/types/src/data.ts

export enum DataCategory {
  HEALTHCARE = 'HEALTHCARE',
  FINANCE = 'FINANCE',
  NLP = 'NLP',
  VISION = 'VISION',
  GEOSPATIAL = 'GEOSPATIAL',
  SCIENTIFIC = 'SCIENTIFIC',
}

export interface DataListingDto {
  category: DataCategory;
  format: string;        // 'parquet' | 'csv' | 'jsonl'
  sizeGb: number;
  metadataHash: string;
  qualityScore: number;  // 0-100
  pricePerAccess: string;
  privacyProof: string;  // hex-encoded differential privacy proof
}

export interface DataBidDto {
  category: DataCategory;
  formatRequirements: string[];
  sizeRange: { min: number; max: number };
```

```
  maxPrice: string;
  qualityThreshold: number;
}
```

Inference Types Shared

```
// packages/types/src/inference.ts

export interface InferenceJobDto {
  modelId: number;
  encryptedInput: string;    // hex-encoded encrypted payload
  maxPrice: string;
  maxLatencyMs: number;
  throughputRequirement: number;  // tokens/second
}

export interface ModelRegistryEntry {
  id: number;
  name: string;
  version: string;
  requiredGpu: GpuType;
  benchmarkScore: number;
  isOpenSource: boolean;
  developer: string;  // address
}
```

Derivatives Types Shared

```
// packages/types/src/derivatives.ts

export interface PerpetualPosition {
  id: string;
  trader: string;
  gpuType: GpuType;
  side: OrderSide;
```

```
  size: string;
  entryPrice: string;
  markPrice: string;
  margin: string;
  leverage: number;
  unrealizedPnl: string;
  liquidationPrice: string;
}

export interface OptionContract {
  id: string;
  type: 'CALL' | 'PUT';
  gpuType: GpuType;
  strikePrice: string;
  expiry: string;
  premium: string;
  holder: string;
}

export interface ForwardContract {
  id: string;
  gpuType: GpuType;
  quantity: number;
  forwardPrice: string;
  maturityDate: string;
  settlementType: 'PHYSICAL' | 'CASH';
  tokenAddress: string;
}
```

Agent Types Shared

```
// packages/types/src/agent.ts

export interface AgentIdentity {
  id: number;
  capabilityHash: string;
  reputationScore: number;
  registeredAt: string;
  isActive: boolean;
}
```

```
export interface AgentCreditScore {
  agentId: number;
  score: number;           // 0-1000
  creditLimit: string;     // max transaction value in USDC
  totalTransactions: number;
  successRate: number;     // 0-100%
}

export interface AgentTask {
  id: string;
  agentId: number;
  taskCommitment: string;  // hash of task specification
  outputCommitment: string;
  proof: string;            // Proof of Agent Work
  verified: boolean;
  completedAt: string;
}
```

Redis Pub/Sub Event Protocol Shared

```
// packages/types/src/events.ts

export type RedisChannel =
  | 'adp:events:batch'
  | 'adp:events:order'
  | 'adp:events:settlement'
  | 'adp:events:market'
  | 'adp:events:provider'
  | 'adp:events:data'
  | 'adp:events:inference'
  | 'adp:events:derivatives'
  | 'adp:events:agent';

export interface RedisEvent<T> {
  channel: RedisChannel;
  event: string;
  payload: T;
```

```
  timestamp: number;
}
```

SECTION 11

Master Timeline

Complete development timeline across all six phases with developer ownership markers.

Phase 0 — Core Dark Pool

Weeks 1–10

Weeks 1–2

Project Scaffolding & Contract Stubs

Sorrowz DarkPool.sol, Escrow.sol, SettlementVerifier.sol stubs and interfaces

Powerz TokenRegistry.sol, ComputeCredit.sol, FeeCollector.sol stubs; monorepo setup

Weeks 3–4

Core Contract Implementation

Sorrowz Commit-reveal scheme, batch lifecycle, escrow deposit/release logic

Powerz Token registry, fee collection, compute credit mint/burn, deployment scripts

Weeks 5–6

Backend Services

Sorrowz Orders module, matching engine implementation, settlement service

Powerz Auth module (SIWE), provider module, market module, WebSocket server

Weeks 7–8

ZK Integration & Indexer

Sorrowz Noir circuit for settlement proof, verifier integration

Powerz Event indexer, Redis pub/sub, WebSocket event relay

Weeks 9–10

Frontend Integration & Testing

Sorrowz Order flow pages, auction timer, settlement history

Powerz Market overview, provider dashboard, settings, live feed

Phase 1 — Private Data Marketplace

Q1–Q2 2027

Q1 2027

Data Marketplace Foundation

Sorrowz Data bid matching engine, DataAccessToken.sol, escrow extension

Powerz Encrypted data listings service, DataProvenance.sol (ERC-721 + ERC-2981)

Q2 2027

Privacy & Token Integration

Sorrowz Federated access layer, long-window escrow settlement

Powerz Differential privacy Noir circuits, verifier contract, token staking tiers

Phase 2 — Private AI Inference Marketplace

Q3–Q4 2027

Q3 2027

Inference Infrastructure

Sorrowz InferencePool.sol, encrypted job submission, inference auctions

Powerz ModelRegistry.sol, ZK proof of correct execution circuit and verifier

Q4 2027

Validation & Agent Integration

Sorrowz SLA enforcement contracts, dispute resolution, slashing logic

Powerz Validator network, AgentKit integration, Coinbase Paymaster setup

Phase 3 — Private Perpetuals & Derivatives

Q1–Q2 2028

Q1 2028

Sorrowz GPU-hour perpetuals, compute options, forward contracts

Powerz ADP Compute Price Index oracle, InsuranceFund.sol

Q2 2028

Cross-Margin & Institutional

Sorrowz Cross-margin engine, portfolio margining, liquidation engine

Powerz Coinbase Prime integration, Base lending composability, liquidity mining

Phase 4 — Privacy-as-a-Service SDK

Q3 2028 – Q2 2029

Q3–Q4 2028

SDK Core Modules

Sorrowz @adp/zk-orders npm package, BaaS REST API + WebSocket

Powerz @adp/tee-compute SDK, @adp/zk-circuits template library

Q1–Q2 2029

Compliance & Developer Experience

Sorrowz BaaS configuration API, multi-asset auction support, revenue share

Powerz Privacy compliance layer, Coinbase Verifications integration, docs portal

Phase 5 — Agentic Economy Infrastructure

Q3 2029+

Q3–Q4 2029

Agent Identity & Marketplace

Sorrowz Agent dark pool marketplace, agent-specific order types

Powerz AgentIdentity.sol (ZK-based), AgentCredit.sol, Coinbase Verifications + ERC-8004

Q1 2030+

Coordination & Treasury

Sorrowz AgentCoordinator.sol, multi-agent compute coordination, Proof of Agent Work

Powerz Agent treasury management, x402 deep integration, Base DeFi composability

SECTION 12

Dependency Map

Architecture overview showing how Phase 0 core primitives are reused and extended across all later phases.

System Architecture Diagram

```
                      AGENTIC DARK POOL — FULL ARCHITECTURE
=====================================================================

PHASE 5: AGENTIC ECONOMY                          Q3 2029+
┌──────────────────────────────────────────────────────────────────┐
│  AgentIdentity.sol    AgentCredit.sol    AgentCoordinator.sol    │
│  Agent Dark Pool      Agent Treasury     Proof of Agent Work     │
│  x402 Integration     ERC-8004           Agent Swarm Mgmt        │
└──────────────────────────────────────────────────────────────────┘
                           │ uses
PHASE 4: PRIVACY SDK                              Q3 2028 – Q2 2029
┌──────────────────────────────────────────────────────────────────┐
│  @adp/zk-orders        @adp/tee-compute     @adp/zk-circuits    │
│  Batch Auction aaS      TEE Orchestration    Privacy Compliance  │
└──────────────────────────────────────────────────────────────────┘
                           │ packages
PHASE 3: DERIVATIVES                              Q1–Q2 2028
┌──────────────────────────────────────────────────────────────────┐
│  GPU Perpetuals         Compute Options      Forward Contracts   │
│  Cross-Margin Engine    Price Index Oracle    Insurance Fund     │
│  Liquidation Engine     Liquidity Mining      Coinbase Prime     │
└──────────────────────────────────────────────────────────────────┘
                           │ extends
PHASE 2: AI INFERENCE                             Q3–Q4 2027
┌──────────────────────────────────────────────────────────────────┐
│  InferencePool.sol      ModelRegistry.sol    SLA Enforcement     │
│  Inference Auctions     ZK Correct Exec      Validator Network   │
│  Encrypted Jobs         AgentKit Integ.      Coinbase Paymaster  │
└──────────────────────────────────────────────────────────────────┘
                           │ extends
PHASE 1: DATA MARKETPLACE                         Q1–Q2 2027
┌──────────────────────────────────────────────────────────────────┐
│  Data Bid Matching      DataProvenance.sol   DataAccessToken.sol │
│  Federated Access       Encrypted Listings   Diff. Privacy Proofs │
│  Long-Window Escrow     OnchainKit Mint      Staking Tiers       │
└──────────────────────────────────────────────────────────────────┘
                           │ extends
PHASE 0: CORE DARK POOL                           Weeks 1–10
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
```

AGENTIC DARK POOL — INTERNAL DEVELOPMENT DOCUMENT — CONFIDENTIAL — VERSION 2.0 — MARCH 29, 2026

```
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐        │
│  │ DarkPool.sol │  │  Escrow.sol  │  │ SettlementVerifier │       │
│  │ Commit-Reveal│  │ USDC Lock    │  │ ZK Proof Verify    │       │
│  └──────┬───────┘  └──────┬───────┘  └──────┬─────────────┘       │
│         │                 │                 │                     │
│  ┌──────┴─────────────────┴─────────────────┴──────────────┐      │
│  │              MATCHING ENGINE (Batch Auction)            │      │
│  │         Uniform Price Clearing + ZK Proof Gen           │      │
│  └─────────────────────────┬───────────────────────────────┘      │
│                            │                                      │
│  ┌──────────┐  ┌──────────┴──┐  ┌──────────┐  ┌──────────┐      │
│  │  Auth    │  │  Orders     │  │ Provider │  │  Market  │      │
│  │  Module  │  │  Module     │  │  Module  │  │  Module  │      │
│  └──────────┘  └─────────────┘  └──────────┘  └──────────┘      │
│                                                                   │
│  ┌──────────────────┐  ┌──────────────────────────┐              │
│  │  WebSocket Server │  │  Event Indexer (on-chain)  │            │
│  │  Socket.io + Redis │  │  viem + PostgreSQL         │            │
│  └──────────────────┘  └──────────────────────────┘              │
│                                                                   │
│  ┌────────────────┐  ┌────────────────┐  ┌────────────────┐      │
│  │ TokenRegistry  │  │ ComputeCredit  │  │  FeeCollector  │      │
│  └────────────────┘  └────────────────┘  └────────────────┘      │
└──────────────────────────────────────────────────────────────────┘

INFRASTRUCTURE LAYER
┌──────────────────────────────────────────────────────────────────┐
│  PostgreSQL 16  │  Redis 7  │  Base (Sepolia/Mainnet)  │  Noir  │
└──────────────────────────────────────────────────────────────────┘
```

Critical Integration Points

INTEGRATION	FROM	TO	PHASE	OWNER
Commit-reveal → Matching engine	DarkPool.sol	Matching Engine	0	Sorrowz
Settlement proof → Verifier	Settlement Service	SettlementVerifier.sol	0	Sorrowz
Fee collection on settlement	DarkPool.sol	FeeCollector.sol	0	Powerz
On-chain events → Indexer	All contracts	Event Indexer	0	Powerz

INTEGRATION	FROM	TO	PHASE	OWNER
Data bid matching extends compute auction	Matching Engine	Data Bid Engine	1	Sorrowz
Data provenance → OnchainKit	DataProvenance.sol	OnchainKit	1	Powerz
Inference auctions extend batch engine	Matching Engine	Inference Auctions	2	Sorrowz
Correct execution proof → SLA	ZK Verifier	SLA Enforcement	2	Both
Price index feeds derivatives	ADP Price Index	Perpetuals/Options	3	Both
Cross-margin uses all position types	Spot + Derivs	Margin Engine	3	Sorrowz
SDK packages core primitives	Phase 0-2 code	@adp/* packages	4	Both
Agent identity → Dark pool	AgentIdentity.sol	Agent Dark Pool	5	Both
Proof of Agent Work → Credit	PoAW Verifier	AgentCredit.sol	5	Both

SECTION 13

Token Utility Implementation

Complete token utility map across all phases with implementation ownership.

Utility by Phase

PHASE	UTILITY	DESCRIPTION	OWNER
0	Protocol Governance	Vote on fee rates, batch intervals, supported GPU types	Powerz
0	Fee Discounts	Token holders receive reduced trading fees (up to 50% discount)	Sorrowz
0	Staking Rewards	Stake tokens to earn share of protocol fees	Sorrowz
1	Data Access Tiers	Higher stake unlocks premium dataset access levels	Sorrowz
1	Data Provider Rewards	Token rewards distributed to data providers based on usage	Powerz
1	Data Standards Governance	Vote on data quality standards and category definitions	Powerz
2	Inference Credits	Stake tokens to receive inference credit allocation	Sorrowz
2	Validator Rewards	Proof verification nodes earn tokens per validated job	Powerz
2	Model Staking	Stake on model quality; earn rewards for popular models	Powerz
3	Derivatives Fee Discounts	Token holders get reduced fees on perpetuals and options	Sorrowz

PHASE	UTILITY	DESCRIPTION	OWNER
3	Liquidity Mining	Token rewards for market makers on derivatives markets	Powerz
3	Insurance Fund Staking	Stake tokens as insurance backstop; earn liquidation fees	Sorrowz
3	Index Governance	Vote on price index methodology and data sources	Powerz
4	SDK Access Fees	Token payment for premium SDK features and higher rate limits	Sorrowz
4	Circuit Marketplace	Buy and sell custom ZK circuits; fees in tokens	Powerz
4	Revenue Share	SDK users share revenue with token stakers	Sorrowz
5	Agent Staking	Agents stake tokens to participate in agent dark pool	Sorrowz
5	Agent Reputation Mining	Tokens earned by agents for successful task completion	Powerz
5	Agent Treasury Yield	Token staking yield for agent treasuries	Powerz
All	Burn Mechanism	Percentage of all fees permanently burned; deflationary pressure	Sorrowz

ⓘ **Implementation Split:** Sorrowz owns all fee mechanics, staking logic, and the burn mechanism across contracts. Powerz owns reward distribution algorithms, governance voting, and identity-related token mechanics.

SECTION 14

Base Ecosystem Integration Map

All Coinbase and Base ecosystem integrations mapped by phase and responsible developer.

Integration Matrix

INTEGRATION	PHASE	PURPOSE	OWNER
x402 Payment Protocol	0	Machine-to-machine payments for compute	Powerz
AgentKit	0	AI agent wallet and transaction management	Powerz
Smart Wallets	0	Gasless transactions, session keys for traders	Powerz
OnchainKit	0	Frontend components: wallet connect, transaction UI	Sorrowz
Coinbase Verifications	1	Identity attestations for data provider KYC	Powerz
Base Data API	1	On-chain data queries for data marketplace indexing	Powerz
Coinbase Paymaster	2	Sponsored gas for inference job submissions	Powerz
Base Lending Protocols	3	Collateral composability for derivatives	Powerz
Coinbase Prime	3	Institutional custody and derivatives access	Powerz
Base Developer Docs	4	SDK documentation hosting and developer guides	Powerz

INTEGRATION	PHASE	PURPOSE	OWNER
Coinbase Ventures	4	Strategic partnership and ecosystem funding	Sorrowz
ERC-8004 / AgentKit v2	5	Agent identity standard and autonomous operations	Powerz
Base Ecosystem Fund	5	Ecosystem grants for agent economy builders	Sorrowz

SECTION 15

Revenue Projections

Projected annual revenue by phase based on market sizing and fee structures.

Revenue by Phase

PHASE	REVENUE SOURCE	FEE RATE	PROJECTED ANNUAL REVENUE
Phase 0	Spot compute trading fees	0.2%	$2M – $8M
Phase 1	Data marketplace fees	1 – 2%	$5M – $20M
Phase 2	Inference marketplace fees	0.5%	$10M – $40M
Phase 3	Derivatives trading fees	0.05%	$15M – $60M
Phase 4	SDK licensing and revenue share	Variable	$8M – $30M
Phase 5	Agent economy transaction fees	Variable	$20M – $100M+

ⓘ **Total Addressable Revenue at Full Scale:** $60M – $258M+ annually across all phases. Revenue compounds as each phase builds on the liquidity and infrastructure of previous phases.

SECTION 16

Appendix & Sign-off

Final notes, acknowledgements, and developer acceptance.

Document Conventions

BADGE	MEANING
Sorrowz	Work item owned by Sorrowz (Core Trading & Settlement Lead)
Powerz	Work item owned by Powerz (Infrastructure & Services Lead)
Shared	Joint ownership; both developers collaborate on integration
Phase N	Identifies the development phase for a work item

Acceptance

By referencing this document, both developers acknowledge the work split and commit to delivering their assigned modules within the specified timelines. Cross-review is mandatory for all pull requests touching shared interfaces or integration points.

Sorrowz

Core Trading & Settlement Lead

Signature: _____

Date: _____

Powerz

Infrastructure & Services Lead

Signature: _____

Date: _____

AGENTIC DARK POOL — INTERNAL DEVELOPMENT DOCUMENT — CONFIDENTIAL — VERSION 2.0 — MARCH 29, 2026